Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 6: Python Functions

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Write Your First Function
2. Parameters & Arguments
3. Types of Arguments
4. *args (Arbitrary Positional Arguments)
5. **kwargs (Arbitrary Keyword Arguments)
6. Function Execution & Return
7. Variable Scope
8. Nested Functions
9. Functions as First-Class Citizens
10. Lambda Functions
11. Higher-Order Functions
12. map()
13. filter()
14. reduce()

84 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 6: Python Functions - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Write Your First Function, Parameters & Arguments, Types of Arguments, *args (Arbitrary Positional Arguments), **kwargs (Arbitrary Keyword Arguments), Function Execution & Return, Variable Scope, Nested Functions, Functions as First-Class Citizens, Lambda Functions, Higher-Order Functions, map(), filter(), reduce()
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Write Your First Function

Q1. Welcome Message

BEGINNER

Write a function welcome() that prints "Welcome to Bikkad IT". Call it two times.

Solution:

```
def welcome():
    print("Welcome to Bikkad IT")

welcome()
welcome()
```

Output:

```
Welcome to Bikkad IT
Welcome to Bikkad IT
```

Q2. Square of a Number

BEGINNER

Write a function square(n) that returns n x n. Print the square of 7 and 12.

Solution:

```
def square(n):
    return n * n

print(square(7))
print(square(12))
```

Output:

```
49
144
```

Q3. Even or Odd Function

INTERMEDIATE

Write a function is_even(num) that returns True if a number is even, else False. Test with 10 and 7.

Solution:

```
def is_even(num):
    return num % 2 == 0

print(is_even(10))
print(is_even(7))
```

Output:

```
True
False
```

Q4. Function with Docstring

INTERMEDIATE

Write a function area_of_circle(r) with a docstring explaining what it does. Return the area rounded to 2 decimals. Print the result for r = 7 and print the docstring using __doc__.

Solution:

```
import math

def area_of_circle(r):
    """Return the area of a circle of radius r (2 decimals)."""
    return round(math.pi * r * r, 2)

print(area_of_circle(7))
print(area_of_circle.__doc__)
```

Output:

```
153.94
Return the area of a circle of radius r (2 decimals).
```

Q5. Reusable GST Calculator

ADVANCED

Write a function add_gst(amount, rate) that returns the final amount. Use it for three products: 1000 at 18%, 250 at 5% and 5000 at 28%.

Solution:

```
def add_gst(amount, rate):
    gst = amount * rate / 100
    return amount + gst

print(add_gst(1000, 18))
print(add_gst(250, 5))
print(add_gst(5000, 28))
```

Output:

```
1180.0
262.5
6400.0
```

Q6. Prime Checker Function

ADVANCED

Write a function is_prime(n) and use it to print all prime numbers between 1 and 30.

Solution:

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True

for num in range(1, 31):
    if is_prime(num):
        print(num, end=" ")
print()
```

Output:

```
2 3 5 7 11 13 17 19 23 29
```

2. Parameters & Arguments

Q7. Greet by Name

BEGINNER

Write greet(name) that prints "Hello" with the name. Call it with "Sneha" and "Rohan". Identify the parameter and the arguments in comments.

Solution:

```
def greet(name):          # name is the PARAMETER
    print("Hello", name)

greet("Sneha")            # "Sneha" is an ARGUMENT
greet("Rohan")
```

Output:

```
Hello Sneha
Hello Rohan
```

Q8. Rectangle Area

BEGINNER

Write area(length, width) that returns the area. Call it with 12 and 8.

Solution:

```
def area(length, width):
    return length * width

print(area(12, 8))
```

Output:

```
96
```

Q9. Simple Interest

INTERMEDIATE

Write simple_interest(p, r, t) returning P x R x T / 100. Calculate for 50000 at 8% for 3 years.

Solution:

```
def simple_interest(p, r, t):
    return p * r * t / 100

si = simple_interest(50000, 8, 3)
print(si)
```

Output:

```
12000.0
```

Q10. Missing Argument Error

INTERMEDIATE

Call area(length, width) with only one argument inside try-except and print the error message.

Solution:

```
def area(length, width):
    return length * width

try:
    print(area(10))
except TypeError as e:
    print(e)
```

Output:

```
area() missing 1 required positional argument: 'width'
```

Q11. Electricity Bill Function

ADVANCED

Write bill(units, rate1, rate2, limit) where units up to limit use rate1 and the rest use rate2. Calculate the bill for 260 units with rates 5 and 8 and limit 100.

Solution:

```
def bill(units, rate1, rate2, limit):
    if units <= limit:
        return units * rate1
    return limit * rate1 + (units - limit) * rate2

print(bill(260, 5, 8, 100))
print(bill(80, 5, 8, 100))
```

Output:

```
1780
400
```

Q12. Validate Arguments

ADVANCED

Write divide(a, b) that returns a / b but prints "Cannot divide by zero" and returns None when b is 0. Test both cases.

Solution:

```
def divide(a, b):
    if b == 0:
        print("Cannot divide by zero")
        return None
    return a / b

print(divide(45, 9))
print(divide(45, 0))
```

Output:

```
5.0
Cannot divide by zero
None
```

3. Types of Arguments

Q13. Default Argument

BEGINNER

Write order_tea(sugar="Normal") that prints the sugar level. Call it with no argument and with "Less".

Solution:

```
def order_tea(sugar="Normal"):
    print("Tea with", sugar, "sugar")

order_tea()
order_tea("Less")
```

Output:

```
Tea with Normal sugar
Tea with Less sugar
```

Q14. Positional Arguments

BEGINNER

Write intro(name, city) and call it with ("Amit", "Pune"). Then swap the order and observe the wrong output.

Solution:

```
def intro(name, city):
    print(name, "lives in", city)

intro("Amit", "Pune")
intro("Pune", "Amit")
```

Output:

```
Amit lives in Pune
Pune lives in Amit
```

Q15. Keyword Arguments

INTERMEDIATE

Call intro(name, city) using keyword arguments in any order: intro(city="Goa", name="Sara").

Solution:

```
def intro(name, city):
    print(name, "lives in", city)

intro(city="Goa", name="Sara")
```

Output:

```
Sara lives in Goa
```

Q16. Mix Positional, Keyword and Default

INTERMEDIATE

Write book_ticket(name, seat, food="No"). Call it as book_ticket("Om", "12A") and book_ticket("Pia", seat="3C", food="Veg").

Solution:

```
def book_ticket(name, seat, food="No"):
    print(name, seat, food)

book_ticket("Om", "12A")
book_ticket("Pia", seat="3C", food="Veg")
```

Output:

```
Om 12A No
Pia 3C Veg
```

Q17. EMI with Default Rate

ADVANCED

Write emi(principal, years, rate=9.5) using EMI = P*r*(1+r)^n / ((1+r)^n - 1) where r = rate/12/100, n = years*12. Print EMI for 500000 over 5 years with default rate, and with rate 8.

Solution:

```
def emi(principal, years, rate=9.5):
    r = rate / 12 / 100
    n = years * 12
    value = principal * r * (1 + r) ** n / ((1 + r) ** n - 1)
    return round(value, 2)

print(emi(500000, 5))
print(emi(500000, 5, rate=8))
```

Output:

```
10500.93
10138.2
```

Q18. Mutable Default Trap

ADVANCED

Write add_item(item, cart=[]) that appends and returns the cart. Call it twice and see the surprise. Then fix it using cart=None.

Solution:

```
def add_item(item, cart=[]):
    cart.append(item)
    return cart

print(add_item("Pen"))
print(add_item("Book"))

def add_item_fixed(item, cart=None):
    if cart is None:
        cart = []
    cart.append(item)
    return cart

print(add_item_fixed("Pen"))
print(add_item_fixed("Book"))
```

Output:

```
['Pen']
['Pen', 'Book']
['Pen']
['Book']
```

4. *args (Arbitrary Positional Arguments)

Q19. Add Any Numbers

BEGINNER

Write total(*args) that returns the sum of any number of values. Test with (10, 20) and (5, 15, 25, 35).

Solution:

```
def total(*args):
    return sum(args)

print(total(10, 20))
print(total(5, 15, 25, 35))
```

Output:

```
30
80
```

Q20. What is args?

BEGINNER

Write show(*args) that prints args and its type. Call it with "Pen", "Book", "Bag".

Solution:

```
def show(*args):
    print(args)
    print(type(args))

show("Pen", "Book", "Bag")
```

Output:

```
('Pen', 'Book', 'Bag')
<class 'tuple'>
```

Q21. Average of Marks

INTERMEDIATE

Write average(*marks) that returns the average rounded to 2 decimals, or 0 if no marks are given.

Solution:

```
def average(*marks):
    if len(marks) == 0:
        return 0
    return round(sum(marks) / len(marks), 2)

print(average(78, 85, 91))
print(average())
```

Output:

```
84.67
0
```

Q22. Fixed + Variable Arguments

INTERMEDIATE

Write bill(customer, *items) where items are prices. Print the customer name, number of items and total.

Solution:

```
def bill(customer, *items):
    print(customer)
    print(len(items))
    print(sum(items))

bill("Ravi", 120, 450, 80)
```

Output:

```
Ravi
3
650
```

Q23. Unpack a List into *args

ADVANCED

Prices are stored in a list [99, 250, 499]. Call total(*args) by unpacking the list with *.

Solution:

```
def total(*args):
    return sum(args)

prices = [99, 250, 499]
print(total(*prices))
```

Output:

```
848
```

Q24. Largest Value Without max()

ADVANCED

Write largest(*nums) that finds the largest number using a loop (no max()). Return None if nothing is passed.

Solution:

```
def largest(*nums):
    if not nums:
        return None
    big = nums[0]
    for n in nums[1:]:
        if n > big:
            big = n
    return big

print(largest(45, 92, 17, 63))
print(largest())
```

Output:

```
92
None
```

5. **kwargs (Arbitrary Keyword Arguments)

Q25. Print a Profile

BEGINNER

Write profile(**kwargs) that prints every key and value. Call it with name="Isha", age=22, city="Pune".

Solution:

```
def profile(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

profile(name="Isha", age=22, city="Pune")
```

Output:

```
name Isha
age 22
city Pune
```

Q26. What is kwargs?

BEGINNER

Write show(**kwargs) that prints kwargs and its type.

Solution:

```
def show(**kwargs):
    print(kwargs)
    print(type(kwargs))

show(brand="Dell", ram=16)
```

Output:

```
{'brand': 'Dell', 'ram': 16}
<class 'dict'>
```

Q27. Order with Options

INTERMEDIATE

Write pizza(size, **toppings) that prints the size and then each topping with its quantity.

Solution:

```
def pizza(size, **toppings):
    print(size)
    for t, qty in toppings.items():
        print(t, qty)

pizza("Large", cheese=2, olives=1, corn=1)
```

Output:

```
Large
cheese 2
olives 1
corn 1
```

Q28. Unpack a Dictionary with **

INTERMEDIATE

A settings dictionary {"theme": "dark", "font": 14} must be passed to apply(theme, font). Use ** to unpack it.

Solution:

```
def apply(theme, font):
    print(theme, font)

settings = {"theme": "dark", "font": 14}
apply(**settings)
```

Output:

```
dark 14
```

Q29. args and kwargs Together

ADVANCED

Write report(title, *scores, **info) that prints the title, average of scores and all extra info.

Solution:

```
def report(title, *scores, **info):
    print(title)
    print(sum(scores) / len(scores))
    for k, v in info.items():
        print(k, v)

report("Unit Test", 70, 80, 90, teacher="Mr. Rao", term=1)
```

Output:

```
Unit Test
80.0
teacher Mr. Rao
term 1
```

Q30. Build SQL-like Filter Text

ADVANCED

Write make_filter(**conditions) that returns a string like "city='Pune' AND age=25" from the keyword arguments.

Solution:

```
def make_filter(**conditions):
    parts = []
    for col, val in conditions.items():
        if isinstance(val, str):
            parts.append(f"{col}='{val}'")
        else:
            parts.append(f"{col}={val}")
    return " AND ".join(parts)

print(make_filter(city="Pune", age=25))
```

Output:

```
city='Pune' AND age=25
```

6. Function Execution & Return

Q31. Function Without return

BEGINNER

Write show_total(a, b) that only prints the sum (no return). Store its result in a variable and print it to see None.

Solution:

```
def show_total(a, b):
    print(a + b)

result = show_total(10, 20)
print(result)
```

Output:

```
30
None
```

Q32. print vs return

BEGINNER

Write add_print(a, b) that prints and add_return(a, b) that returns. Try to multiply each result by 2.

Solution:

```
def add_print(a, b):
    print(a + b)

def add_return(a, b):
    return a + b

x = add_return(3, 4) * 2
print(x)

y = add_print(3, 4)
print(y)
```

Output:

```
14
7
None
```

Q33. Return Multiple Values

INTERMEDIATE

Write min_max(numbers) that returns both the minimum and maximum. Unpack and print them for [42, 17, 88, 63].

Solution:

```
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([42, 17, 88, 63])
print(low)
print(high)
```

Output:

```
17
88
```

Q34. Code After return Does Not Run

INTERMEDIATE

Write check_age(age) that returns "Adult" if age >= 18 and "Minor" otherwise. Put a print after the first return to show it never runs.

Solution:

```
def check_age(age):
    if age >= 18:
        return "Adult"
        print("This line never runs")
    return "Minor"

print(check_age(20))
print(check_age(15))
```

Output:

```
Adult
Minor
```

Q35. Order of Execution (Stack)

ADVANCED

Write three functions a(), b(), c() where a calls b and b calls c. Print messages when each function starts and ends to see the call stack order.

Solution:

```
def c():
    print("c start")
    print("c end")

def b():
    print("b start")
    c()
    print("b end")

def a():
    print("a start")
    b()
    print("a end")

a()
```

Output:

```
a start
b start
c start
c end
b end
a end
```

Q36. Recursive Factorial

ADVANCED

Write a recursive function fact(n) and print fact(5). Add a print to show each call going into the stack.

Solution:

```
def fact(n):
    print("calling fact", n)
    if n <= 1:
        return 1
    return n * fact(n - 1)

print(fact(5))
```

Output:

```
calling fact 5
calling fact 4
calling fact 3
calling fact 2
calling fact 1
120
```

7. Variable Scope

Q37. Local Variable

BEGINNER

Inside a function make_tea(), create a local variable sugar = 2 and print it. Try to print sugar outside the function inside try-except.

Solution:

```
def make_tea():
    sugar = 2
    print(sugar)

make_tea()

try:
    print(sugar)
except NameError as e:
    print(e)
```

Output:

```
2
name 'sugar' is not defined
```

Q38. Read a Global Variable

BEGINNER

A global variable shop_name = "Bikkad Cafe" is used inside a function bill_header() to print the header.

Solution:

```
shop_name = "Bikkad Cafe"

def bill_header():
    print(shop_name)

bill_header()
```

Output:

```
Bikkad Cafe
```

Q39. Same Name, Different Variables

INTERMEDIATE

Create a global x = 10 and inside a function a local x = 99. Print x inside and outside the function.

Solution:

```
x = 10

def show():
    x = 99
    print(x)

show()
print(x)
```

Output:

```
99
10
```

Q40. Change a Global with global

INTERMEDIATE

A visitor counter is global count = 0. Write visit() that increases it using the global keyword. Call it 3 times.

Solution:

```
count = 0

def visit():
    global count
    count += 1

visit()
visit()
visit()
print(count)
```

Output:

```
3
```

Q41. UnboundLocalError

ADVANCED

Write a function that tries count += 1 on a global without using global, inside try-except, and print the error.

Solution:

```
count = 0

def visit():
    count += 1

try:
    visit()
except UnboundLocalError as e:
    print(e)
```

Output:

```
cannot access local variable 'count' where it is not associated
with a value
```

Q42. Better: Pass and Return

ADVANCED

Rewrite the visitor counter WITHOUT global: write visit(count) that returns count + 1, and update the variable outside.

Solution:

```
def visit(count):
    return count + 1

count = 0
count = visit(count)
count = visit(count)
count = visit(count)
print(count)
```

Output:

```
3
```

8. Nested Functions

Q43. Function Inside a Function

BEGINNER

Write outer() that defines inner() which prints "Inner function". outer() should call inner().

Solution:

```
def outer():
    print("Outer function")

    def inner():
        print("Inner function")

    inner()

outer()
```

Output:

```
Outer function
Inner function
```

Q44. Inner Function is Hidden

BEGINNER

Try to call inner() from outside outer() inside try-except and print the error.

Solution:

```
def outer():
    def inner():
        print("Inner function")
    inner()

try:
    inner()
except NameError as e:
    print(e)
```

Output:

```
name 'inner' is not defined
```

Q45. Helper for a Bill

INTERMEDIATE

Write final_bill(amount) that has an inner function gst(x) returning 18% of x. Return amount + gst(amount).

Solution:

```
def final_bill(amount):
    def gst(x):
        return x * 18 / 100
    return amount + gst(amount)

print(final_bill(1000))
```

Output:

```
1180.0
```

Q46. Inner Function Reads Outer Variable

INTERMEDIATE

Write greet(name) with an inner function message() that uses name from the outer function.

Solution:

```
def greet(name):
    def message():
        return "Good morning " + name
    return message()

print(greet("Kavya"))
```

Output:

```
Good morning Kavya
```

Q47. nonlocal Counter

ADVANCED

Write counter() with a local count = 0 and an inner increment() that uses nonlocal to increase it. Call increment 3 times inside counter and return count.

Solution:

```
def counter():
    count = 0

    def increment():
        nonlocal count
        count += 1

    increment()
    increment()
    increment()
    return count

print(counter())
```

Output:

```
3
```

Q48. Validate then Process

ADVANCED

Write register(name, age) with two inner helper functions: valid_name() and valid_age(). Only print "Registered" if both are valid.

Solution:

```
def register(name, age):
    def valid_name():
        return name.isalpha() and len(name) >= 2

    def valid_age():
        return 18 <= age <= 60

    if valid_name() and valid_age():
        print("Registered", name)
    else:
        print("Invalid details")

register("Ravi", 25)
register("R2", 25)
```

Output:

```
Registered Ravi
Invalid details
```

9. Functions as First-Class Citizens

Q49. Assign Function to a Variable

BEGINNER

Write say_hi() and assign it to a variable greet. Call greet(). Print the type of say_hi.

Solution:

```
def say_hi():
    print("Hi!")

greet = say_hi
greet()
print(type(say_hi))
```

Output:

```
Hi!
<class 'function'>
```

Q50. Store Functions in a List

BEGINNER

Write add(a, b) and sub(a, b). Store them in a list and call each with (10, 4).

Solution:

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

operations = [add, sub]
for op in operations:
    print(op(10, 4))
```

Output:

```
14
6
```

Q51. Pass a Function as Argument

INTERMEDIATE

Write apply(func, value) that returns func(value). Use it with len on "Python" and with abs on -25.

Solution:

```
def apply(func, value):
    return func(value)

print(apply(len, "Python"))
print(apply(abs, -25))
```

Output:

```
6
25
```

Q52. Calculator with Dictionary of Functions

INTERMEDIATE

Create a dictionary {"+": add, "-": sub, "*": mul}. For the operator "*" and numbers 6, 7 call the correct function.

Solution:

```
def add(a, b):
    return a + b

def sub(a, b):
    return a - b

def mul(a, b):
    return a * b

calc = {"+": add, "-": sub, "*": mul}
op = "*"
print(calc[op](6, 7))
```

Output:

```
42
```

Q53. Return a Function (Factory)

ADVANCED

Write make_multiplier(n) that returns a function which multiplies by n. Create double and triple and use them.

Solution:

```
def make_multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(15))
print(triple(15))
```

Output:

```
30
45
```

Q54. Delete a Function

ADVANCED

Write temp() and also assign it to backup. Delete temp with del. Show that temp() fails but backup() still works.

Solution:

```
def temp():
    print("temp running")

backup = temp
del temp

try:
    temp()
except NameError as e:
    print(e)

backup()
```

Output:

```
name 'temp' is not defined
temp running
```

10. Lambda Functions

Q55. Square with Lambda

BEGINNER

Write a lambda that returns the square of a number and use it for 9.

Solution:

```
square = lambda x: x * x
print(square(9))
```

Output:

```
81
```

Q56. Add Two Numbers

BEGINNER

Write a lambda add that takes two numbers and returns the sum. Use it for 15 and 27.

Solution:

```
add = lambda a, b: a + b
print(add(15, 27))
```

Output:

```
42
```

Q57. Lambda with if-else

INTERMEDIATE

Write a lambda result that returns "Pass" if marks >= 35 else "Fail". Test with 42 and 20.

Solution:

```
result = lambda m: "Pass" if m >= 35 else "Fail"

print(result(42))
print(result(20))
```

Output:

```
Pass
Fail
```

Q58. Sort with Lambda Key

INTERMEDIATE

Sort students [("Om", 72), ("Pia", 95), ("Raj", 81)] by marks (high to low) using a lambda key.

Solution:

```
students = [("Om", 72), ("Pia", 95), ("Raj", 81)]

ranked = sorted(students, key=lambda s: s[1], reverse=True)
print(ranked)
```

Output:

```
[('Pia', 95), ('Raj', 81), ('Om', 72)]
```

Q59. Lambda vs def

ADVANCED

Write the same discount logic (10% off) using def and lambda. Print both results and their __name__ values to see the difference.

Solution:

```
def discount_def(price):
    return price * 0.9

discount_lambda = lambda price: price * 0.9

print(discount_def(500))
print(discount_lambda(500))
print(discount_def.__name__)
print(discount_lambda.__name__)
```

Output:

```
450.0
450.0
discount_def
<lambda>
```

Q60. Sort Dictionary Items by Two Keys

ADVANCED

Employees [("Om", "IT", 50000), ("Pia", "HR", 50000), ("Raj", "IT", 65000)]. Sort by salary (high to low) and then by name (A-Z) using one lambda key.

Solution:

```
emps = [("Om", "IT", 50000),
        ("Pia", "HR", 50000),
        ("Raj", "IT", 65000)]

result = sorted(emps, key=lambda e: (-e[2], e[0]))
for e in result:
    print(e)
```

Output:

```
('Raj', 'IT', 65000)
('Om', 'IT', 50000)
('Pia', 'HR', 50000)
```

11. Higher-Order Functions

Q61. Function Takes a Function

BEGINNER

Write do_twice(func) that calls func two times. Pass a function that prints "Beep".

Solution:

```
def do_twice(func):
    func()
    func()

def beep():
    print("Beep")

do_twice(beep)
```

Output:

```
Beep
Beep
```

Q62. Apply to Every Item

BEGINNER

Write apply_all(func, items) that returns a new list with func applied to each item. Use it with str.upper on ["pen", "cup"].

Solution:

```
def apply_all(func, items):
    result = []
    for i in items:
        result.append(func(i))
    return result

print(apply_all(str.upper, ["pen", "cup"]))
```

Output:

```
['PEN', 'CUP']
```

Q63. Custom Price Rules

INTERMEDIATE

Write final_price(price, rule) where rule is a function. Apply a festival rule (20% off) and a member rule (minus 50) to 500.

Solution:

```
def final_price(price, rule):
    return rule(price)

festival = lambda p: p * 0.8
member = lambda p: p - 50

print(final_price(500, festival))
print(final_price(500, member))
```

Output:

```
400.0
450
```

Q64. Return a Greeting Function

INTERMEDIATE

Write get_greeter(language) that returns a different greeting function for "en" and "hi".

Solution:

```
def get_greeter(language):
    if language == "hi":
        return lambda name: "Namaste " + name
    return lambda name: "Hello " + name

greet_hi = get_greeter("hi")
greet_en = get_greeter("en")
print(greet_hi("Om"))
print(greet_en("Om"))
```

Output:

```
Namaste Om
Hello Om
```

Q65. Compose Two Functions

ADVANCED

Write compose(f, g) that returns a function computing f(g(x)). Compose "add 10" after "double" and test with 5.

Solution:

```
def compose(f, g):
    return lambda x: f(g(x))

double = lambda x: x * 2
add10 = lambda x: x + 10

double_then_add = compose(add10, double)
print(double_then_add(5))
```

Output:

```
20
```

Q66. Retry Helper

ADVANCED

Write run_with_retry(func, times) that calls func until it returns True or the attempts run out. Use a list of results [False, False, True] to simulate a payment gateway.

Solution:

```
results = [False, False, True]

def payment():
    return results.pop(0)

def run_with_retry(func, times):
    for attempt in range(1, times + 1):
        if func():
            print("Success on attempt", attempt)
            return True
        print("Attempt", attempt, "failed")
    return False

run_with_retry(payment, 3)
```

Output:

```
Attempt 1 failed
Attempt 2 failed
Success on attempt 3
```

12. map()

Q67. Celsius to Fahrenheit

BEGINNER

Convert temperatures [30, 25, 40] from Celsius to Fahrenheit using map() and a lambda (F = C x 9/5 + 32).

Solution:

```
celsius = [30, 25, 40]

fahrenheit = list(map(lambda c: c * 9 / 5 + 32, celsius))
print(fahrenheit)
```

Output:

```
[86.0, 77.0, 104.0]
```

Q68. Strings to Integers

BEGINNER

Marks were read as strings ["78", "85", "92"]. Convert them to integers with map(int, ...) and find the total.

Solution:

```
marks_text = ["78", "85", "92"]

marks = list(map(int, marks_text))
print(marks)
print(sum(marks))
```

Output:

```
[78, 85, 92]
255
```

Q69. Apply GST with map

INTERMEDIATE

Add 18% GST to prices [100, 250, 400] using map with a normal function.

Solution:

```
def add_gst(p):
    return round(p * 1.18, 2)

prices = [100, 250, 400]
print(list(map(add_gst, prices)))
```

Output:

```
[118.0, 295.0, 472.0]
```

Q70. map with Two Lists

INTERMEDIATE

Quantities [2, 5, 1] and rates [40, 10, 300]. Use map with two lists to get the amount for each item.

Solution:

```
qty = [2, 5, 1]
rate = [40, 10, 300]

amounts = list(map(lambda q, r: q * r, qty, rate))
print(amounts)
```

Output:

```
[80, 50, 300]
```

Q71. Clean User Inputs

ADVANCED

Names entered are [" om ", "PIA", "raj "]. Use map to strip spaces and convert to title case.

Solution:

```
names = ["  om ", "PIA", "raj  "]

clean = list(map(lambda n: n.strip().title(), names))
print(clean)
```

Output:

```
['Om', 'Pia', 'Raj']
```

Q72. Read Numbers from One Line

ADVANCED

A user types "10 20 30 40" in one line. Use input().split() with map(int) to get a list of numbers and print the sum.

Solution:

```
numbers = list(map(int, input("Enter numbers: ").split()))

print(numbers)
print(sum(numbers))
```

Output (with sample input):

```
Enter numbers: 10 20 30 40
[10, 20, 30, 40]
100
```

13. filter()

Q73. Even Numbers

BEGINNER

Filter even numbers from [3, 8, 12, 7, 20, 15] using filter() and lambda.

Solution:

```
numbers = [3, 8, 12, 7, 20, 15]

evens = list(filter(lambda n: n % 2 == 0, numbers))
print(evens)
```

Output:

```
[8, 12, 20]
```

Q74. Passed Students

BEGINNER

Filter marks >= 35 from [28, 67, 45, 12, 90].

Solution:

```
marks = [28, 67, 45, 12, 90]

passed = list(filter(lambda m: m >= 35, marks))
print(passed)
```

Output:

```
[67, 45, 90]
```

Q75. Remove Empty Strings

INTERMEDIATE

A form returned ["Om", "", "Pia", "", "Raj"]. Remove empty entries using filter(None, ...).

Solution:

```
entries = ["Om", "", "Pia", "", "Raj"]

clean = list(filter(None, entries))
print(clean)
```

Output:

```
['Om', 'Pia', 'Raj']
```

Q76. Filter with a Named Function

INTERMEDIATE

Write is_valid_email(e) that checks for "@" and ".". Filter ["a@x.com", "bad.com", "c@y.in"].

Solution:

```
def is_valid_email(e):
    return "@" in e and "." in e

emails = ["a@x.com", "bad.com", "c@y.in"]
print(list(filter(is_valid_email, emails)))
```

Output:

```
['a@x.com', 'c@y.in']
```

Q77. Filter Dictionaries

ADVANCED

Products are [{"name": "Pen", "stock": 0}, {"name": "Book", "stock": 12}, {"name": "Bag", "stock": 3}]. Filter products that are in stock and print their names.

Solution:

```
products = [{"name": "Pen", "stock": 0},
            {"name": "Book", "stock": 12},
            {"name": "Bag", "stock": 3}]

available = filter(lambda p: p["stock"] > 0, products)
for p in available:
    print(p["name"])
```

Output:

```
Book
Bag
```

Q78. filter + map Together

ADVANCED

From prices [120, 450, 80, 999, 300], keep prices above 100 and then apply 10% discount, in one expression.

Solution:

```
prices = [120, 450, 80, 999, 300]

result = list(map(lambda p: p * 0.9,
                  filter(lambda p: p > 100, prices)))
print(result)
```

Output:

```
[108.0, 405.0, 899.1, 270.0]
```

14. reduce()

Q79. Sum with reduce

BEGINNER

Find the sum of [10, 20, 30, 40] using functools.reduce and a lambda.

Solution:

```
from functools import reduce

numbers = [10, 20, 30, 40]
total = reduce(lambda a, b: a + b, numbers)
print(total)
```

Output:

```
100
```

Q80. Product of Numbers

BEGINNER

Find the product of [1, 2, 3, 4, 5] using reduce.

Solution:

```
from functools import reduce

product = reduce(lambda a, b: a * b, [1, 2, 3, 4, 5])
print(product)
```

Output:

```
120
```

Q81. Largest Using reduce

INTERMEDIATE

Find the largest number in [45, 92, 17, 63] using reduce (without max()).

Solution:

```
from functools import reduce

numbers = [45, 92, 17, 63]
largest = reduce(lambda a, b: a if a > b else b, numbers)
print(largest)
```

Output:

```
92
```

Q82. Reduce with Initial Value

INTERMEDIATE

Find the total cart value of [("Pen", 10), ("Book", 60), ("Bag", 450)] using reduce with an initial value of 0.

Solution:

```
from functools import reduce

cart = [("Pen", 10), ("Book", 60), ("Bag", 450)]
total = reduce(lambda acc, item: acc + item[1], cart, 0)
print(total)
```

Output:

```
520
```

Q83. Join Words into a Sentence

ADVANCED

Join ["Python", "is", "powerful"] into one sentence using reduce, and print each step to see how reduce works.

Solution:

```
from functools import reduce

def join(a, b):
    print("joining:", a, "+", b)
    return a + " " + b

sentence = reduce(join, ["Python", "is", "powerful"])
print(sentence)
```

Output:

```
joining: Python + is
joining: Python is + powerful
Python is powerful
```

Q84. map + filter + reduce Pipeline

ADVANCED

Orders are [250, 1200, 80, 560, 999]. Keep orders >= 500, add 18% GST to each, and find the grand total using reduce. Round to 2 decimals.

Solution:

```
from functools import reduce

orders = [250, 1200, 80, 560, 999]

big = filter(lambda o: o >= 500, orders)
with_gst = map(lambda o: o * 1.18, big)
grand_total = reduce(lambda a, b: a + b, with_gst)

print(round(grand_total, 2))
```

Output:

```
3255.62
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026